PRACTICAL WEEK - 7

TASK 1: Write a program that prints the addresses Of code, global, static, heap, and stack variables. Analyze the Linux process address space layout and explain each memory segment

kiran@k: ~/practical_memory

GNU nano 8.7.1

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int global_var = 10;
static int static_var = 20;

void code_function()
{
    printf("This is a code function.\n");
}

int main()
{
    int stack_var = 30;
    static int local_static_var = 40;

    int *heap_var = (int *)malloc(sizeof(int));
    *heap_var = 50;

    printf("Process ID: %d\n", getpid());

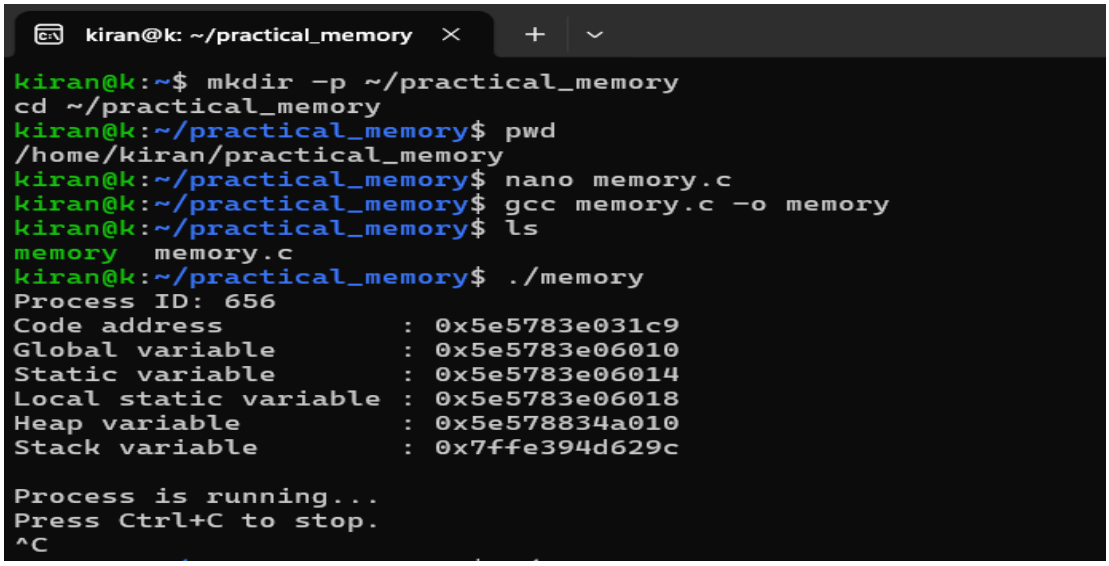
    printf("Code address          : %p\n", (void *)code_function);
    printf("Global variable       : %p\n", (void *)&global_var);
    printf("Static variable       : %p\n", (void *)&static_var);
    printf("Local static variable : %p\n", (void *)&local_static_var);
    printf("Heap variable         : %p\n", (void *)heap_var);
    printf("Stack variable        : %p\n", (void *)&stack_var);

    printf("\nProcess is running...\n");
    printf("Press Ctrl+C to stop.\n");

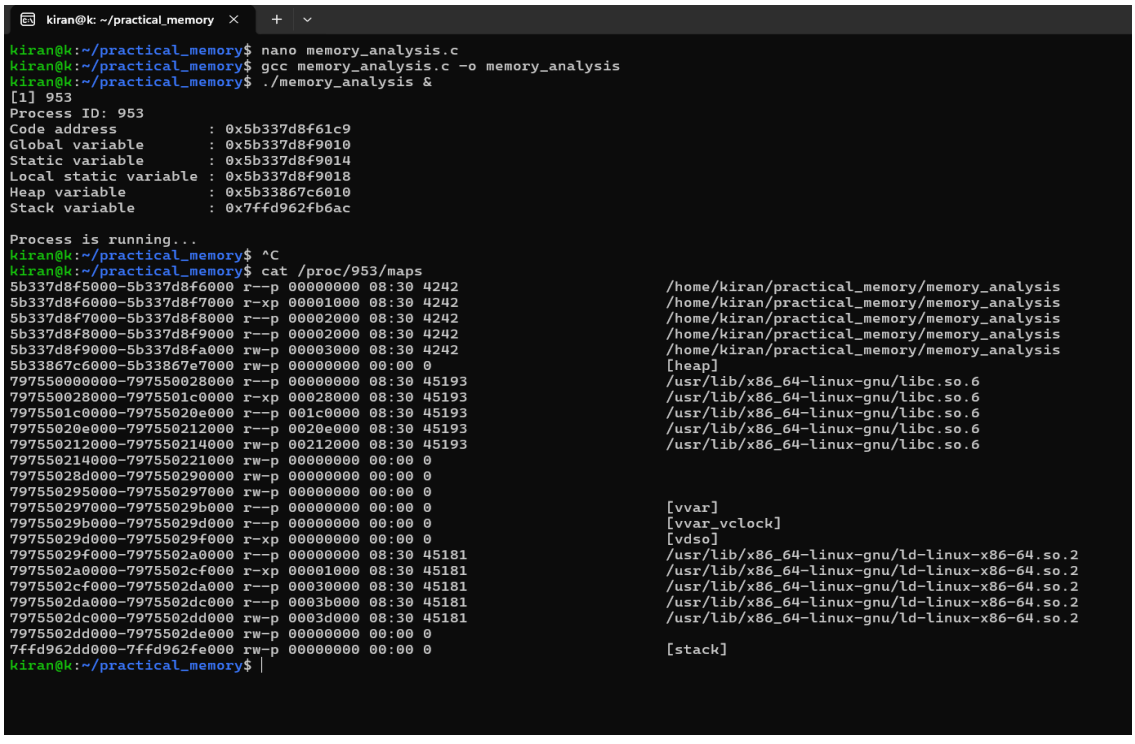
    while (1)
    {
        sleep(2);
    }

    free(heap_var);

    return 0;
}
```




TASK 2: Make use of the /proc//maps file and memory analysis tools to study the memory organization of a running process. Prepare a report explaining virtual memory mappings.



kiran@k: ~/practical_memory

GNU nano 8.7.1 memory_analysis.c *

k: ~/practical_memory
trl+alt+1

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int global_var = 10;
static int static_var = 20;

void code_function()
{
    printf("This is a code function.\n");
}

int main()
{
    int stack_var = 30;
    static int local_static_var = 40;

    int *heap_var = (int *)malloc(sizeof(int));
    *heap_var = 50;

    printf("Process ID: %d\n", getpid());

    printf("Code address          : %p\n", (void *)code_function);
    printf("Global variable       : %p\n", (void *)&global_var);
    printf("Static variable       : %p\n", (void *)&static_var);
    printf("Local static variable : %p\n", (void *)&local_static_var);
    printf("Heap variable         : %p\n", (void *)heap_var);
    printf("Stack variable        : %p\n", (void *)&stack_var);

    printf("\nProcess is running...\n");

    sleep(30);

    free(heap_var);

    return 0;
}
```